

C++ Programming Textbook Notes

Module: Functions in C++ | Prototypes, Definitions & Parameter Passing

Author: Gaurav Kandpal

Platform: CS-Simplified

Level: Beginner to Advanced

1. FUNCTIONS IN C++: CONCEPTUAL FOUNDATIONS

What is a Function?

A **Function** is a named, self-contained block of statements designed to perform a specific, well-defined computational task. Rather than writing duplicate logic repeatedly throughout a codebase, functions allow developers to structure code into modular, reusable components.

Why Use Functions? (The Modular Philosophy)

- **Code Reusability & DRY Principle:** "Don't Repeat Yourself." Write algorithmic logic once and invoke it multiple times across the application.
- **Abstraction & Modularity:** Complex systems are broken down into smaller, manageable sub-problems, making code readable and organized.
- **Ease of Debugging & Testing:** Isolated functions enable targeted unit testing, making bug localization simpler and faster.
- **Memory Efficiency:** Function memory is allocated dynamically on the Call Stack during execution and automatically released upon task completion.

How Functions Work in Memory (The Call Stack Architecture)

During program execution, the Operating System reserves a region of RAM known as the **Call Stack**. Whenever a function call occurs:

1. A discrete block of RAM called a **Stack Frame (Activation Record)** is allocated on top of the stack.
2. The caller pushes function arguments, local variables, and the CPU *Return Address* into this stack frame.
3. CPU control jumps to the function's instructions. Upon encountering a `return` statement, the stack frame is popped (deallocated), restoring control and execution state back to the main routine.

[CALL STACK MEMORY LAYOUT DURING FUNCTION CALL]

High RAM Address

main() Stack Frame
- Local Variables: a = 5, b = 6, s
sum() Activation Record / Stack Frame
- Pushed Arguments / Formal Parameters
- Return Address to main()
- Local Calculations

Low RAM Address (Stack grows downwards)

2. FUNCTION PROTOTYPES, DECLARATIONS, DEFINITIONS & CALLS

A C++ function workflow involves three core components: **Declaration (Prototype)**, **Definition**, and **Call**.

1. Function Declaration (Prototype)

Informs the compiler about the function's existence, return type, name, and parameter data types *before* it is invoked.

```
return_type function_name(data_type1, data_type2);
```

Rule: Parameter names in declarations are dummy/optional identifiers. Writing `int sum(int, int);` is fully valid syntax.

2. Function Definition

Contains the actual body of statements and executable logic.

```
return_type function_name(type1 param1, type2 param2) { /* body */ }
```

Rule: Parameter names are mandatory in function definitions so statement logic can reference those memory locations.

3. Function Call

Transfers execution flow to the defined function by passing real values (actual arguments).

```
int result = function_name(arg1, arg2);
```

Function Ordering Rules

- **Defined Before Call:** If a function's full definition appears *above* its call site in the source code, an explicit function prototype declaration is optional.
- **Defined After Call:** If a function is defined *below* its call site (e.g., below `main()`), a function declaration above `main()` is **mandatory**. Otherwise, compilation fails with an *"undeclared identifier"* error.

3. ARGUMENT TERMINOLOGY: ACTUAL VS. FORMAL ARGUMENTS

Argument Category	Definition	Location in Code
Actual Arguments	The real variables or literal values passed into the function at the call site.	Inside the function call statement, e.g., <code>sum(a, b);</code> in <code>main()</code> .
Formal Arguments	The placeholder parameter variables defined in the function header to receive passed data.	Inside the function declaration/definition header, e.g., <code>int sum(int x, int y)</code> .

4. PARAMETER PASSING MECHANISMS (THE 3 PILLAR TECHNIQUES)

In C++, formal parameters can be categorized into three distinct variable types, corresponding to three core parameter passing methods:

1. Call by Value (Ordinary Variables)

Concept: When formal arguments are ordinary variables (e.g., `int x`), the actual values are *copied* into separate stack memory slots for the formal parameters.

Memory Impact: Changes made to formal parameters inside the function do **not** affect the original actual variables in `main()` because they occupy completely separate memory addresses.

```
// Call by Value Demonstration
#include <iostream>
using namespace std;

int sum(int, int); // Function Declaration (Parameter names optional)

int main() {
    int a = 5, b = 6; // Actual Arguments
    int s = sum(a, b);
    cout << "Sum is " << s << endl;
    return 0;
}

// Formal arguments x and y are ordinary variables (Copies created)
int sum(int x, int y) {
    return (x + y);
}
```

2. Call by Address (Pointer Variables)

Concept: When formal arguments are pointer variables (e.g., `int *p`), the *memory addresses* of the actual variables (e.g., `&a`, `&b`) are passed into the function.

Memory Impact: Dereferencing pointers (using `*p`) grants the function direct access to modify the original actual memory locations in `main()`.

```
// Call by Address Demonstration
#include <iostream>
using namespace std;

int sum(int*, int*); // Declaration accepting pointer types

int main() {
    int a = 5, b = 6;
    int s = sum(&a, &b); // Passing memory addresses (&)
    cout << "Sum is " << s << endl;
    return 0;
}

// Formal arguments p and q are pointer variables holding RAM addresses
int sum(int *p, int *q) {
    return (*p + *q); // Dereferencing pointers to access values
}
```

3. Call by Reference (Reference Variables)

Concept: When formal arguments are reference variables (e.g., `int &x`), the formal parameters become direct *aliases* for the actual arguments passed at the call site.

Memory Impact: No new memory is created for data copies or pointer storage. Modifying formal parameters alters original actual variables directly, with clean syntax omitting dereference symbols (`*`).

```
// Call by Reference Demonstration
#include <iostream>
using namespace std;

int sum(int&, int&); // Declaration accepting reference types

int main() {
    int a = 5, b = 6;
    int s = sum(a, b); // Clean call syntax (looks like call-by-value)
    cout << "Sum is " << s << endl;
    return 0;
}

// Formal arguments x and y are reference aliases for actual variables a and b
int sum(int &x, int &y) {
    return (x + y);
}
```

5. COMPREHENSIVE PARAMETER PASSING COMPARISON

Feature	Call by Value	Call by Address	Call by Reference
Formal Type	Ordinary Variables (int x)	Pointer Variables (int *p)	Reference Variables (int &x)
Call Site Syntax	sum(a, b);	sum(&a, &b);	sum(a, b);
Memory Created	Duplicate copy of data created in stack.	Pointers allocated to hold memory addresses.	Zero copy allocation. Direct RAM alias.
Original Value Impact	Original variables remain unmodified.	Original variables can be modified.	Original variables can be modified directly.
Syntax Cleanliness	Simple.	Requires explicit * and & symbols.	Cleanest. No dereferencing needed.

Common Gotcha: Returning References to Local Stack Variables

Never return a reference to a local variable defined inside a function (e.g., `int& getVal() { int temp = 10; return temp; }`). Local variables are destroyed when the stack frame pops, leaving a **Dangling Reference** pointing to invalid RAM!

Best Practice: Modern C++ Function Design

Use **Call by Value** for primitive types (`int`, `char`, `double`) when no modification is needed. Use **Const Reference** (`const Type &`) for large structs, objects, or strings to eliminate expensive copies while guaranteeing read-only safety!